# Copenhagen Cupcakes

## Webshop med bestilling af cupcakes

**Datamatiker, 2. semester E26, Ek-Lyngby**

**Hold**: DAT-FIR-DA-F26A

**Gruppe**: Team B

**Gruppens medlemmer:**



| **Navn:** | **cph-email** | **GitHub** |
|---|---|---|
| Ali Mohammad Kadhim Sulaiman | cph-as856@stud.ek.dk | alisalman0499 |
| Gabriella Amina Kadiga | cph-gk90@stud.ek.dk | Gabbyskii |
| Hannah Marie Fonclara Concillado | cph-hc143@stud.ek.dk | Hanibans |
| Kristina Bengtsson | cph-kd104@stud.ek.dk | KristinaBengtsson |



**Github**: https://github.com/Hanibans/Cupcake-teamb.git

**Video-demo**: https://youtu.be/v3oyz8BfedM

**Projektets varighed**: 2/11-2026 til 12/11-2026
**Aflevering:** 12/11-2026

# Indholdsfortegnelse


Indholdsfortegnelse 2
Indledning 3
- Baggrund 3
- Teknologivalg 3

Krav 3
- Virksomhedens vision 3
- User Stories 4

Domæne model og ER diagram 6
- Domænemodel 6
- ER diagram 7

Mockups 8
- Low-fi: Vores papirprototyper 8
- Hi-fi: Design i Figma 10

Implementation 15
- Håndtering af sessioner 15
- Håndtering af exceptions 16
- Validering af brugerinput 16
- Sikkerhed i forbindelse med login 16
- Design pattern 16

Test 17
- Unit tests 17
- Integrationstests 19
- Manuel Testning 22

Status på implementation 22
- Kundens funktioner 22
- Administratorens funktioner 23
- Sider og design 23
- Fravalgte og manglende funktioner 23
- Fremtidig implementation 24

Proces 24
- Planer for arbejdsform og projektforløb 24
- Forløbet i praksis 24

Refleksion 25
- Arbejdsform og værktøjer 25
- Roller 25
- Kodestandard og DoD 25
- Hvad gik godt og hvad har vi lært? 25


# Indledning

## Baggrund

Copenhagen Cupcakes er et bageri, som gerne vil sælge cupcakes online. Konceptet er at kunden skal kunne sammensætte sin egen cupcake ved at vælge en bund og en topping ud fra et bestemt udvalg, lægge sine cupcakes i indkøbskurven og betale for ordren igennem deres konto hos bageren. Ordrer afhentes derefter i butikken. Kunden skal have en konto og være logget ind for at kunne betale. Derudover skal bageriet kunne tilgå en administrator side hvor de kan se alle ordrer og kunder, så de kan holde styr på ordrens status.

## Teknologivalg

Java 21
IntelliJ IDEA  25.0.3 | 21.0.8 | 25.0.4.1
Javalin 7.2.3
Javalin-rendering-thymeleaf 7.2.3
PostgreSQL 16.2
PostgreSQL JDBC driver 42.7.8
pgAdmin 4 9.18
JUnit-jupiter-api 5.10.2
HikariCP 7.0.2
slf4j-simple 2.0.17
HTML5 og CSS3
Figma (mockups)
Claude Opus 5.5 Medium
Docker Desktop 4.92.0

# Krav

## Virksomhedens vision

Copenhagen Cupcakes ønsker at kunne sælge deres cupcakes online, så kunderne selv kan sammensætte og bestille deres cupcakes hjemmefra og hente dem i butikken. Systemet skal give bageriet flere salg og spare tid i butikken, fordi ordrerne er bestilt og betalt på forhånd. Samtidig skal bageren have et samlet overblik over alle ordrer og kunder et samlet sted for at lette det administrative arbejde.

## User Stories

**US-1 Bestil og betal cupcakes med valgfri bund og top:**
Som kunde kan jeg bestille og betale cupcakes med en valgfri bund og top, sådan at jeg

senere kan køre forbi butikken og hente min ordre uden at stå i kø.
**Acceptkriterier:**

- Kunden kan vælge præcis en bund fra en liste over tilgængelige bunde.
- Kunden kan vælge præcis en top fra en liste over tilgængelige toppe.
- Kunden kan kun gå til betaling med præcis én type cupcake.
- Kunden bliver sendt til en betalingsside når der trykkes på “gå til betaling”
- Kunden får besked om hvis de har nok på saldoen og betalingen er gennemført
- Kundens saldo opdateres i systemet.

**US-2 Oprette konto for at betale og gemme ordre:**
Som kunde kan jeg oprette en konto/profi for at kunne betale og gemme en ordre.
**Acceptkriterier:**

- Kunden skal kunne oprette en profil med e-mail og password hvis de ikke allerede er oprettet
- E-mail skal være unik. Hvis e-mail allerede findes, afvises oprettelsen med en besked.
- Password skal valideres sådan at den overholder sikkerheds kriterierne.(Mindst 8 tegn. Skal indeholde både tal og bogstaver) (*Delvist implementeret)*
- Efter oprettelsen er kunden logget ind.
- En kunde starter med en konto-saldo på 100 kr.
- Kundens profil skal have mulighed for at kunne gemme ordrer informationer

**US-3 Se ordre og pris i indkøbskurv:**
Som kunde kan jeg se mine valgte ordrelinier i en indkøbskurv, så jeg kan se den samlede pris.
**Acceptkriterier:**

- Kurven skal kunne gemme informationer om valgte cupcakes fra kunden sådan at de løbende kan justere deres ordre
- Kunden skal kunne tilføje flere cupcakes til kurven, og hver cupcake kan have sin egen kombination.
- Kurven skal vise antal og pris pr. cupcake.
- Den samlede pris vises på nederste linje. (vises i DKK)
- En tom kurv skal vise en besked om at den er tom.

**US-4 Login, login vises på alle sider:**
Som kunde eller administrator kan jeg logge på systemet med email og kodeord. Når jeg er logget på, skal jeg kunne se min email på hver side (evt. i topmenuen, som

vist på mockup'en).
**Acceptkriterier:**

- Brugeren kan logge ind med email og kodeord.
- Når email eller adgangskode er indtastet forkert får brugeren en besked om dette men kan ikke se hvilken af de to der er blevet skrevet forkert
- Når brugeren er logget ind, vises email i topmenuen på alle sider.
- Uden login har man ikke adgang til checkout, profil eller administrator sider.
- Både kunder og administrator logger ind med samme formular.

**US-5 Admin kan se alle ordrer:**
Som administrator kan jeg se alle ordrer i systemet, så jeg kan se hvad der er blevet bestilt.
**Acceptkriterier:**

- Administratoren har en oversigt over alle ordrer.
- Oversigten viser ordrenummer, dato, ordrestatus, betalingsstatus, kundens email, samlet pris.
- Betalingsstatus markeres hvis ordre er ubetalt.
- Administratoren kan åbne en ordre og se alle dens cupcakes.
- Kunder uden administratorrettigheder kan ikke åbne oversigten.
- Listen sorteres efter sidst ordre øverst.

**US-6 Admin kan se alle kunder:**
Som administrator kan jeg se alle kunder i systemet og deres ordrer, sådan at jeg kan følge op på ordrer og holde styr på mine kunder.
**Acceptkriterier:**

- Administratoren har en liste over kunder med navn og email.
- Administratoren kan åbne og se en kundes ordrer. (*Delvist implementeret*)
- Administratoren kan ændre status på ordrer.
- Listen sorteres efter sidst betalte ordre øverst.

**US-7 Kunden kan fjerne en ordre:**
Som kunde kan jeg fjerne en ordrelinje fra min indkøbskurv, så jeg kan justere min ordre.
**Acceptkriterier:**

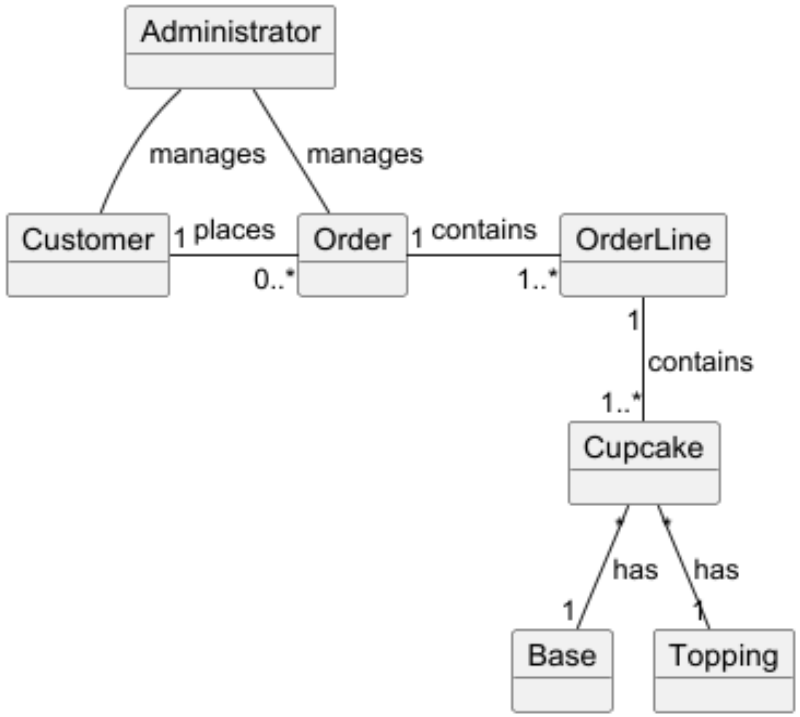
- Kunden kan fjerne eller tilføje flere cupcakes på hovedsiden.
- Kunden kan fjerne eller tilføje flere cupcakes direkte i indkøbskurven.
- Totalen opdateres straks ved ændringer.
- Hvis alle linjer fjernes, vises den tomme kurv besked.

**US-8 Admin kan fjerne en ordre:** *(Ikke implementeret)*
Som administrator kan jeg fjerne en ordre, så systemet ikke kommer til at indeholde ugyldige ordrer. F.eks. hvis kunden aldrig har betalt.
**Acceptkriterier:**

- Administratoren kan slette en ordre.
- Efter sletning vises ordren ikke længere i ordre oversigten eller på kundens profil.
- Sletning kræver en bekræftelse, hvor det er tydeligt, hvilken ordre der slettes.

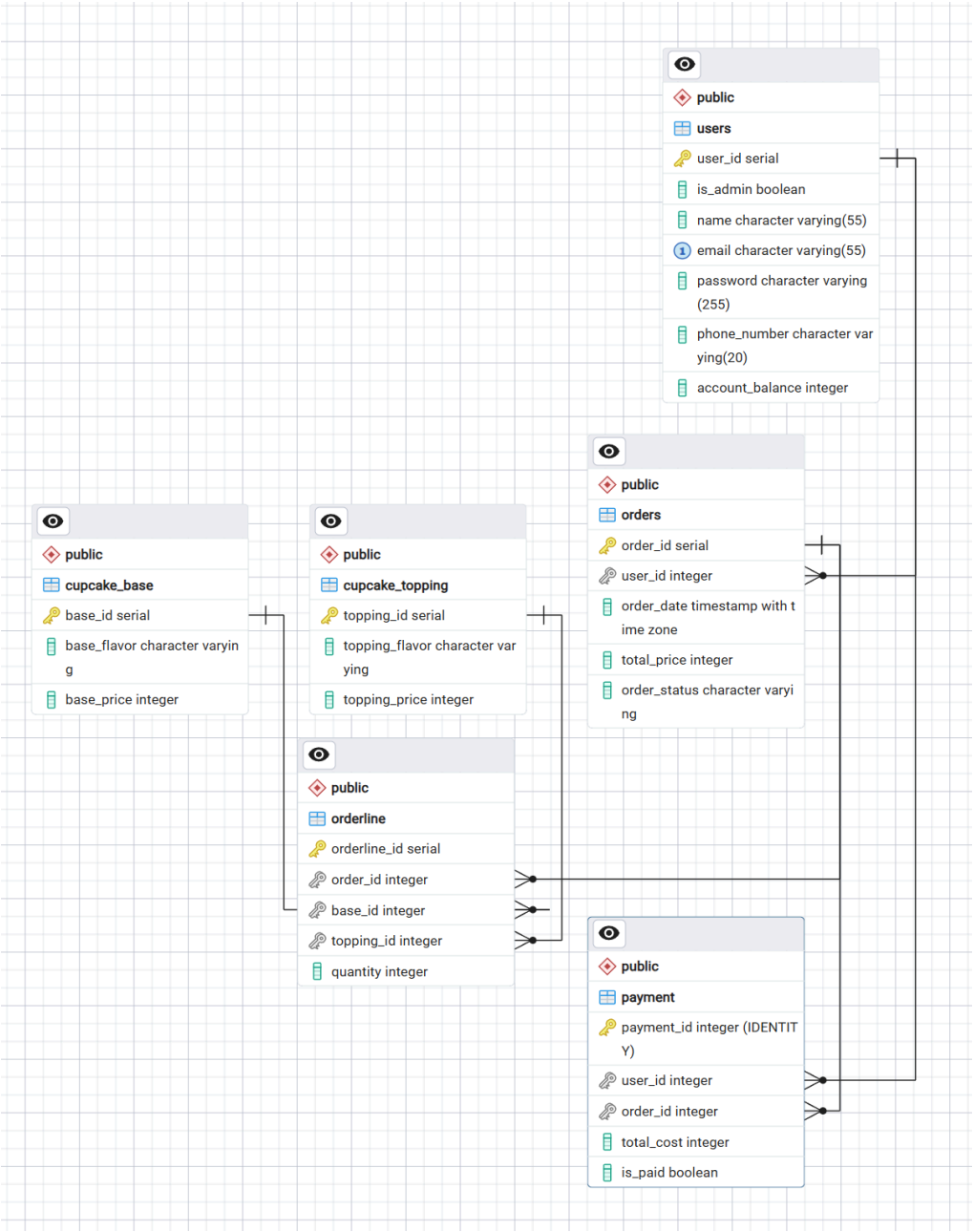
# Domæne model og ER diagram

## Domænemodel




Den relation vi har overvejet mest, er hvordan cupcake hænger sammen med en ordre. Vi valgte at indføre ordrelinjen som et mellemled, fordi en kunde kan bestille flere forskellige cupcakes i samme ordre og flere af hver specifik cupcake, bestående af en base og en topping. En ordrelinje beskriver derfor en slags cupcake og et antal af netop denne cupcake kombination.

## ER diagram




Alle tabeller bruger en automatisk genereret id, som primærnøgle. Email er ikke primærnøgle i users, men den har en UNIQUE constraint, så den samme email ikke kan oprettes to gange.

**Normalisering og bevidste afvigelser**
Databasen overholder overordnet den 3. normalform, med en afvigelse. *orders.total price* kunne beregnes ud fra orderline. Men vi gemmer den alligevel, fordi priserne på bunde og toppings kan ændre sig, og den gemte pris viser, hvad kunden faktisk betalte på købstidspunktet.

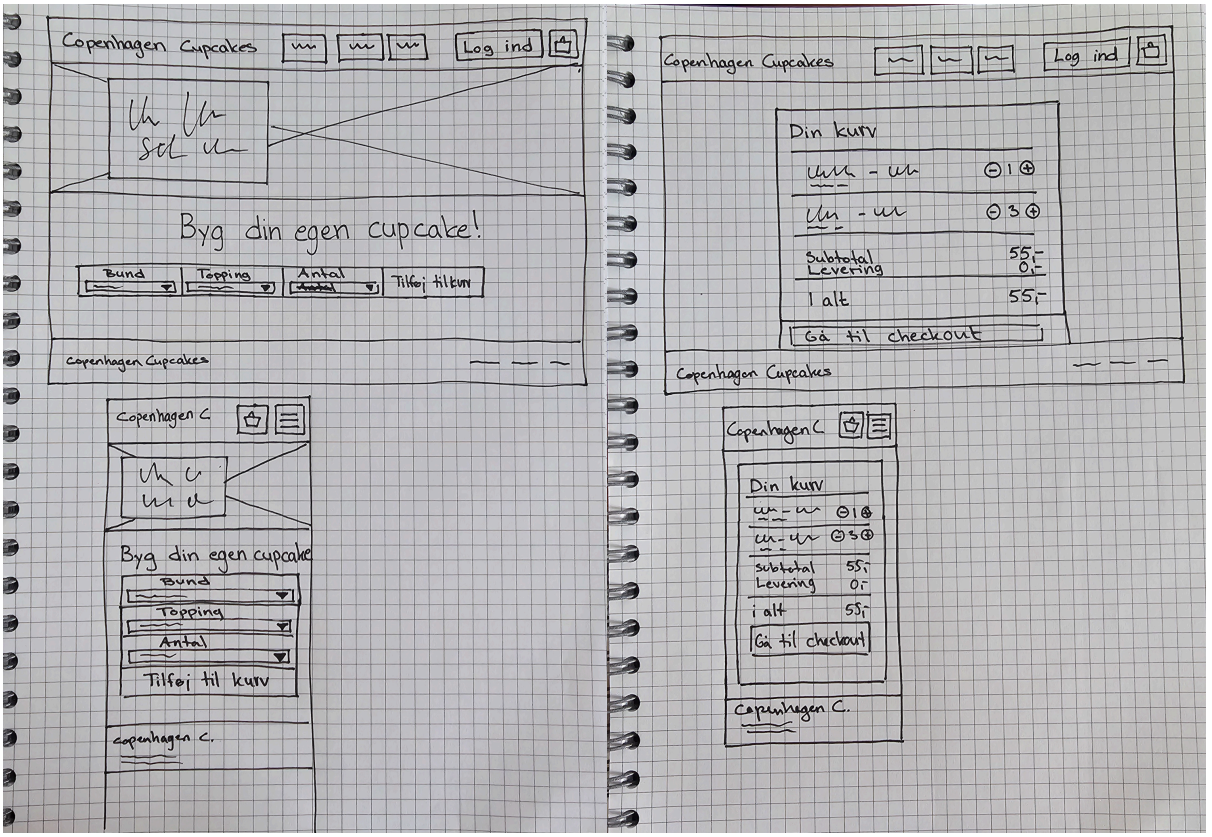
# Mockups

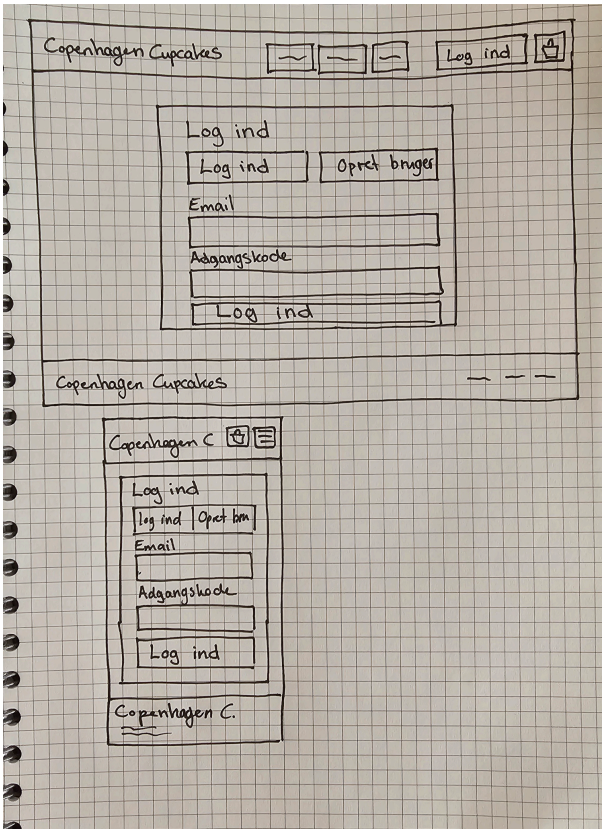
Brugergrænsefladen blev designet i to trin: først low-fi skitser på papir og derefter et hi-fi-design i Figma.

## **Low-fi:** Vores papirprototyper

Vi startede med at tegne mockups på papir af de tre mest centrale sider. Forsiden, indkøbskurven og login. Vi valgte denne løsning fordi det er en hurtig og nem måde at skitsere et grundlæggende design på uden at blive for fokuseret på farver og skrifttyper, det gav os altså mulighed for i første omgang at fokusere på layout og struktur frem for detaljer. Hver side blev tegnet i både en desktop og en mobilversion, fordi opgaven stillede krav om et responsivt design.

Vi baseret vores design med gestaltlovene i tankerne.

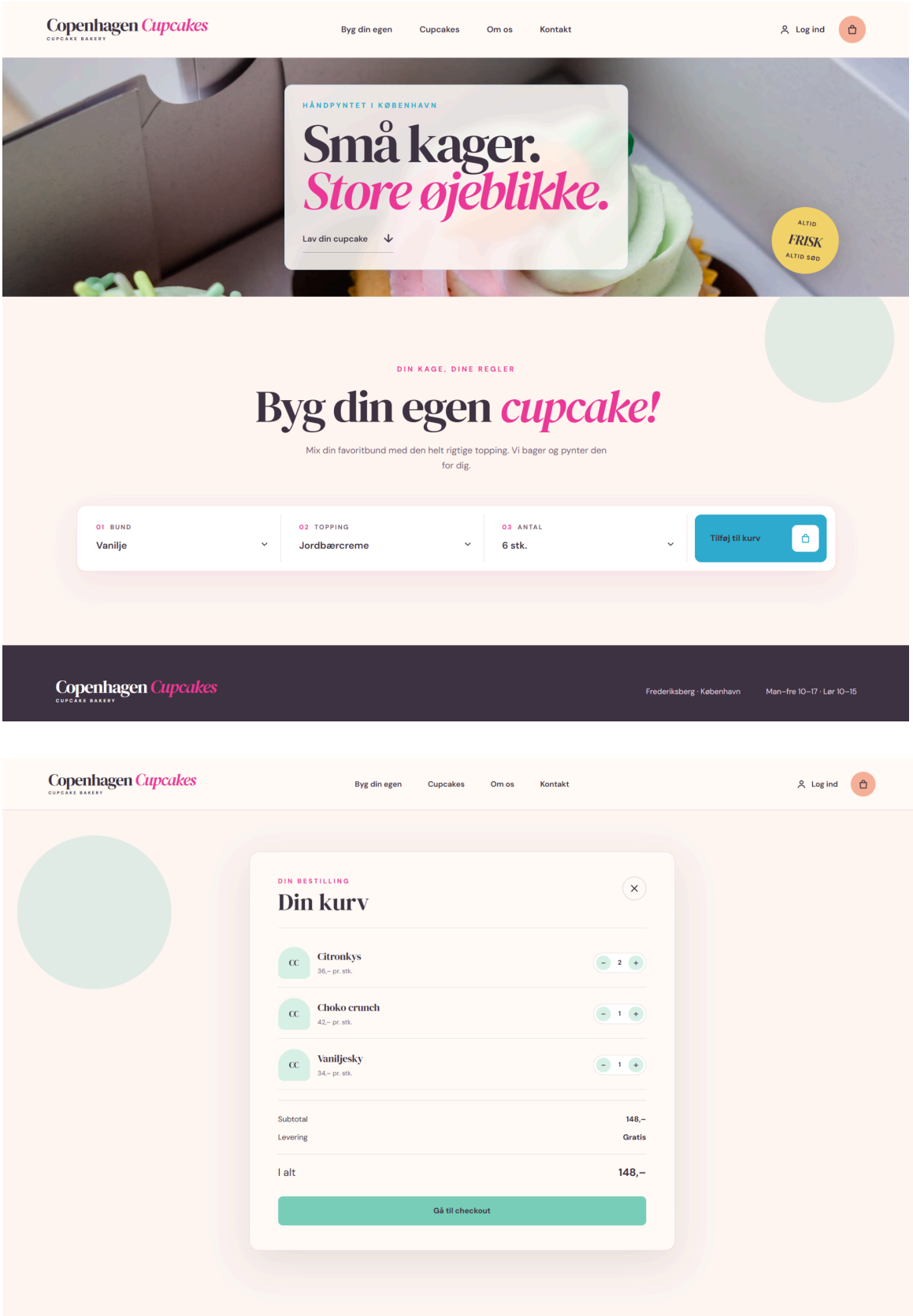





**Nærhed:** Elementer med samme funktion blev grupperet, sådan så menu knapperne står samlet for sig, og kurv samt login er placeret sammen. Vi brugte også luft mellem de forskellige dele af siden, så det er nemt at se hvor en funktion starter og slutter. Vi brugte det også i kurven hvor subtotal og levering står tæt sammen, mens “I alt” er skilt fra med en streg, på den måde ser man tydeligt at det er en sammenlægning af beløbene.

**Lighed:** Menu knapperne i headeren har samme form og størrelse og det samme gælder dropdowns når man vælger bund, topping og antal.

## **Hi-fi:** Design i Figma



Efter vi var tilfredse med papir skitserne, uploadede vi dem sammen med et farveskema vi var enige om at bruge til Figma. Ud fra dette arbejder vi videre med Figmas AI-værktøj, indtil vi nåede frem til det endelige design.

I hi-fi-designet brugte vi farver til at skabe lighed. Elementer med samme funktion har samme farve, så brugeren kan se at de hører sammen. Vi brugte også farvekontraster til at vise, hvilken knap der er aktiv, og hvilken der ikke er. Dette ses fx på login-siden hvor den aktive knap af login og opret bruger står frem og brugeren opmærksomhed bliver styret hen mod det vigtigste.

**Fra design til kode:** Vi er programmører, ikke designere.

Ud fra billederne af Figma-designet fik vi genereret en simpelt HTML- og CSS-version med AI. Som programmører og ikke frontend designere valgte vi med den begrænset tid i projektet, at prioritere vores tid på backend, database og forretningslogik. Vi valgte dog bevidst at den genererede kode ikke indeholdt ThymeLeaf, så vi selv implementerede den dynamiske del. Derudover tilpassede vi selv den genererede kode hvor vi fjernede de elementer vi ikke fandt nødvendige.

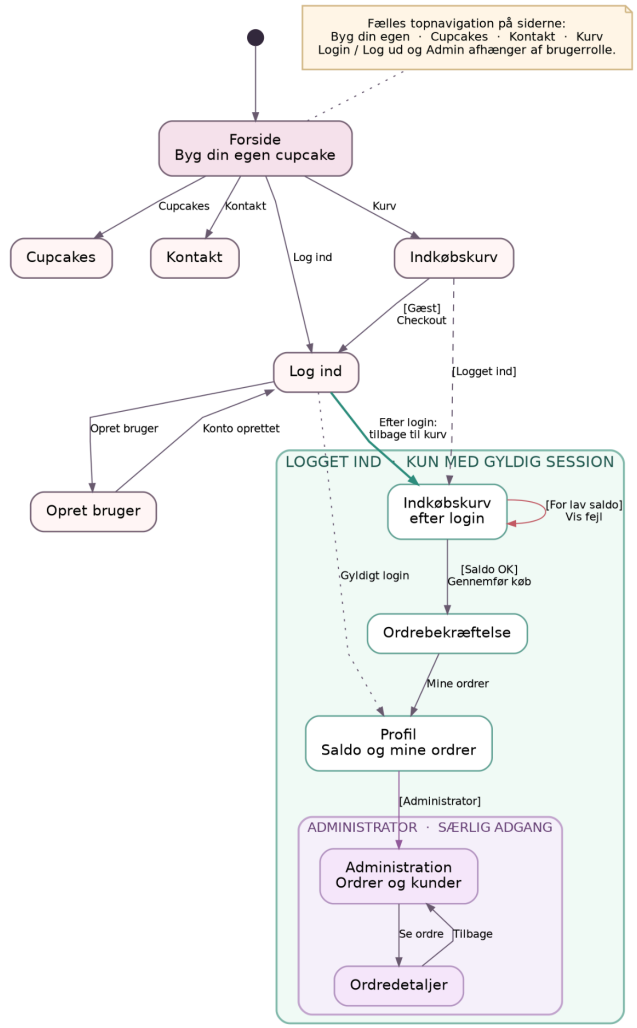
# Navigationsdiagram

I vores projekt har vi valgt at udarbejde et navigationsdiagram med screenshots fra vores hjemmeside for at give et overblik over de forskellige sider og hvordan brugeren kan navigere mellem dem. Da hjemmesiden har forskellige funktioner alt efter om brugeren er logget ind eller ej, har vi valgt at opdele navigationsdiagrammet i mindre dele for at gøre det mere overskueligt.

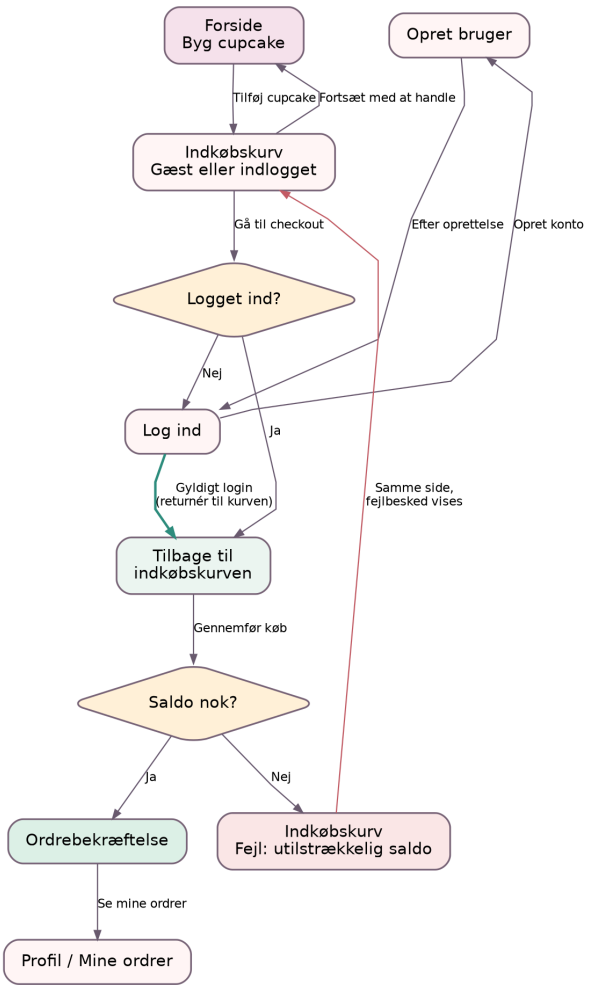
Vi har blandt andet fokuseret på hvordan en bruger kan oprette en konto, logge ind og gennemføre en bestilling. Her har vi også taget højde for de forskellige situationer der kan opstå undervejs, som eksempelvis hvis brugeren forsøger at gennemføre en bestilling uden at være logget ind eller ikke har tilstrækkeligt med penge på saldoen. I disse tilfælde bliver brugeren enten sendt til login-siden eller får vist en fejlmeddelelse i indkøbskurven. Efter login bliver brugeren sendt tilbage til indkøbskurven, så vedkommende kan fortsætte sin bestilling.

Vi har desuden en fælles navigationsbar på hjemmesiden, som giver brugeren mulighed for at bevæge sig mellem de forskellige sider. Navigationsbaren ændrer sig alt efter brugerens rolle, hvor en administrator eksempelvis har adgang til administrationssiden, som en almindelig bruger ikke har. Dette har vi også forsøgt at tydeliggøre i navigationsdiagrammet.

## COPENHAGEN CUPCAKES · NAVIGATIONSOVERSIGT



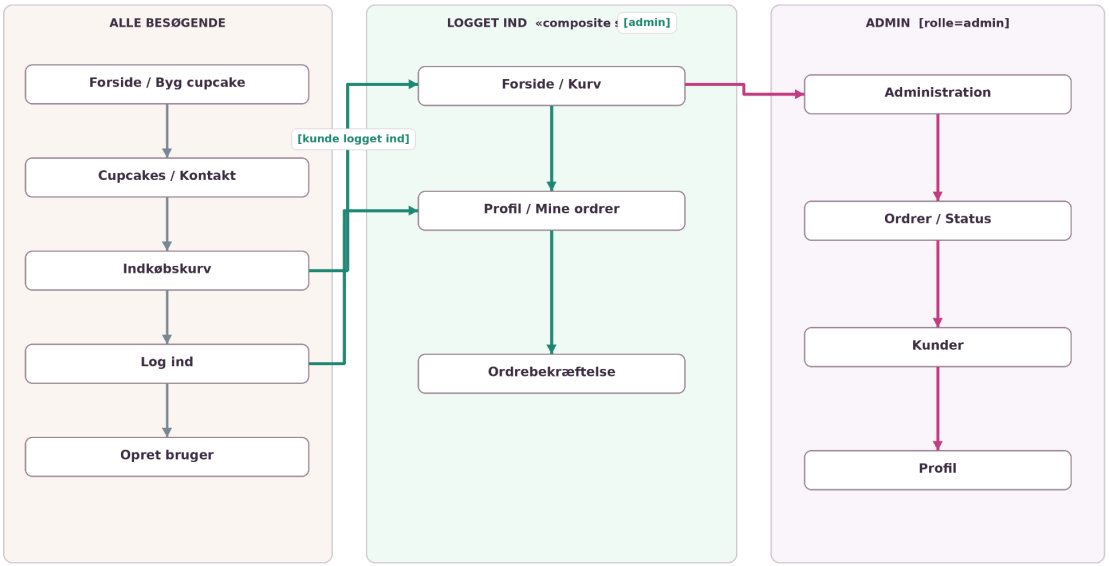

## COPENHAGEN CUPCAKES · BESTILLINGSNAVIGATION



COPENHAGEN CUPCAKES

## 01 · Overordnet navigationsdiagram

Sidernes sammenhæng · Adgang: gæst, kunde og administrator




Alle kan se forside og kurv; checkout kræver login. Adminsider kræver administratorrettigheder.

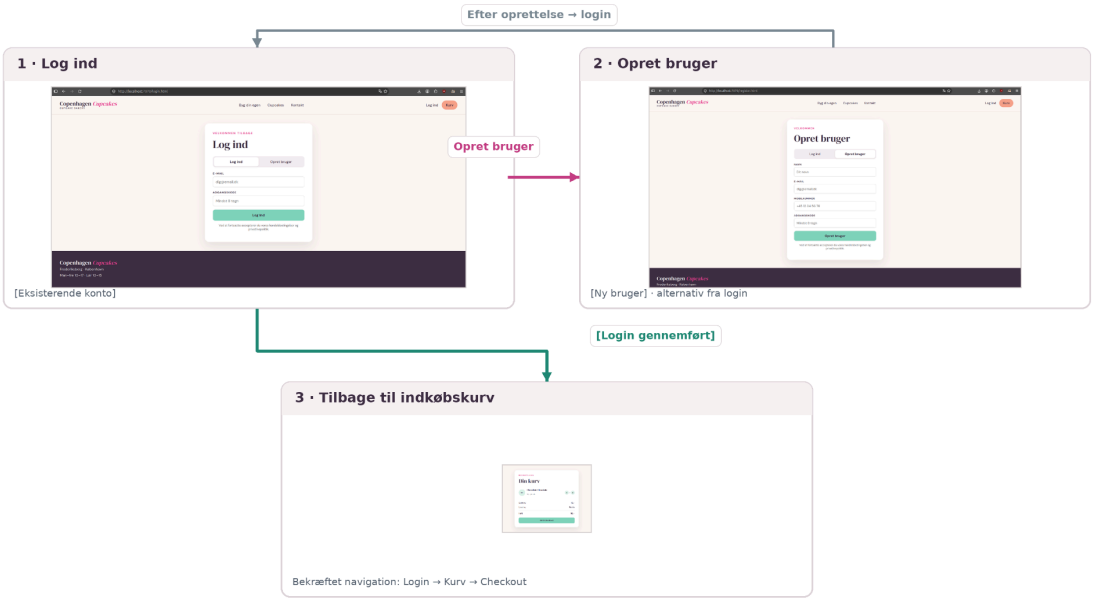
Fælles topmenu: Logo · Byg din egen · Cupcakes · Kontakt · Login/e-mail · Kurv

Screenshots kan findes i højere opløsning i Bilag

COPENHAGEN CUPCAKES

## 02 · Login og oprettelse

Detaljediagram · De faktiske sider fra hjemmesiden




Fra kurven til login · Efter gennemført login vender kunden tilbage til indkøbskurven.

COPENHAGEN CUPCAKES

## 03 · Bestilling gennemført

Scenarie 1 · Ikke logget ind ved checkout; tilstrækkelig saldo efter login

### 1 · Vælg cupcake

Vælg bund, topping og antal

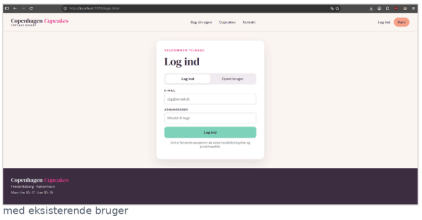
Tilføj til kurv

### 2 · Indkøbskurv

Klik "Gå til checkout"

[ikke logget ind]

### 3 · Login [gæst]




Log ind med eksisterende bruger

[saldo OK]

### 4 · Ordrebekræftelse

[Logget ind + saldo tilstrækkelig]

1 → 2 → 3 → 4 · Betingelsen [tilstrækkelig saldo] giver adgang til ordrebekræftelsen.

COPENHAGEN CUPCAKES

## 04 · Utilstrækkelig saldo

Scenarie 2 · Logget ind, men saldoen rækker ikke til ordren

### 1 · Byg cupcakes

Tilføj flere cupcakes til kurven

Gå til kurv

### 2 · Indkøbskurv

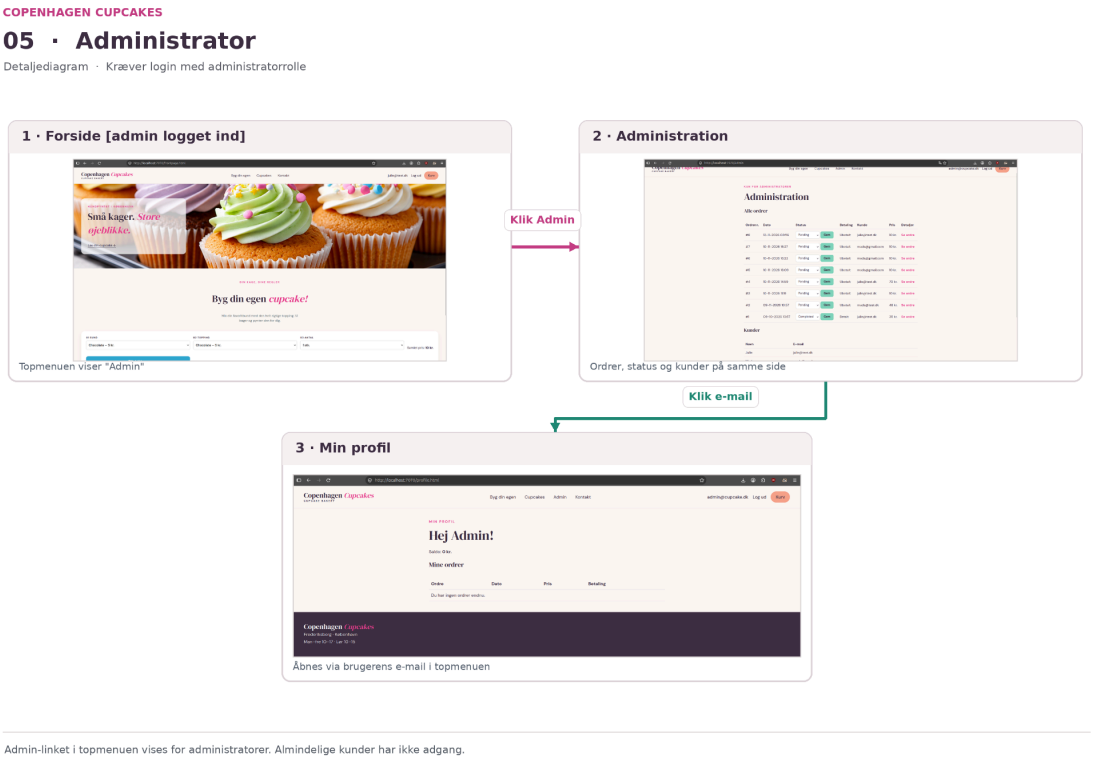
Samlet beløb overstiger saldoen

Checkout [for lav saldo]

### 3 · Samme side med fejlbesked

[saldo utilstrækkelig] · køb afvises

Fejlmeddelelsen vises på indkøbskurven – der åbnes ikke en ny webside.




# Implementation

## Håndtering af sessioner

I UserController gemmes et User-objekt under session-nøglen "user", når en bruger logger ind eller opretter en konto. Objektet oprettes gennem metoder i UserService. Ved logout sættes "user" til null, så brugerens loginstatus fjernes.

I OrderController gemmes kundens kurv under session-nøglen "orderlines" som en ArrayList<Orderline>. Listen opdateres, når kunden tilføjer eller fjerner varer eller ændrer antallet, og tømmes efter en gennemført ordre. Derudover gemmes "selectedOrderId", som holder styr på den ordre, administratoren har valgt. Vi anvender også sessionen til midlertidige ”flash” beskeder. Eksempelvis gemmes "adminMessage", når en administrator opdaterer en ordrestatus. Beskeden hentes efter et redirect ved hjælp af consumeSessionAttribute, så den kun vises én gang. Samme princip anvendes til "errorMessage", som vises, hvis kundens saldo ikke dækker ordren i kurven.

Vi anvender desuden request-attributter i showOrder-metoden, hvor orders, selectedOrder og customers sendes til admin.html med ctx.attribute. Disse

informationer gemmes ikke i sessionen, men hentes fra databasen ved hvert request, da de kun skal bruges til at vise den aktuelle side.

## Håndtering af exceptions

Vi har en exceptions-mappe som håndterer en DatabaseException klasse og en IllegalUserException klasse. Vores exceptions bliver fanget ved try-catch som vi omringer størstedelen af vores mapper- og controller klassers metoder med.

**DatabaseException:** Fanges i controllerne. I checkout-metoden under OrderController klassen returneres status 500 med en fejlbesked, og i renderFrontpage under UserController klassen returneres status 400.

**IllegalUserDataException:** Fanges i createUser og createuserCheckout, som returnerer fejlbeskeden til brugeren.

## Validering af brugerinput

Vi har implementeret minimalt brugervalidering i vores UserService klasse. Vi har tre metoder som sørger for bruger input validering: En validateEmail() som sørger for at email-en har et ”@” symbol. En validatePhoneNumber() som sørger for at telefonnummeret består af præcis 8-cifre. Og en validatePassword() for at sikre at brugerens adgangskode indeholder mellem 8-15 tegn. Alle tre valideringsmetoder bliver kaldt som det første i createUser-metoden, så IllegalUserDataException bliver fanget hvis kriterierne ikke overholdes. Vi kalder hermed en userExists() metode fra UserMapper-klassen i vores createUser() som tjekker at brugeren ikke allerede findes, når man opretter en ny bruger.

## Sikkerhed i forbindelse med login

Vi har implementeret en generel fejlbesked i login-metoden i UserController, som vises via ctx.result, hvis brugeren indtaster en forkert e-mailadresse eller adgangskode. Fejlbeskeden angiver ikke specifikt, hvilken af oplysningerne der er forkert. Dette gør det sværere for uvedkommende at finde ud af, om en bestemt e-mailadresse eksisterer i systemet.

## Design pattern

Vi har anvendt MVC- design pattern til at strukturere projektet. Hermed har vi opdelt backend-logikken i Controller-, Service- og Mapper-lag. Controlleren håndterer requests og kommunikationen mellem View og backend. Service-laget håndterer

applikationens forretningslogik, mens Mapper-laget står for kommunikationen med databasen og håndtering af data.

# Test

I vores projekt har vi testet manuelt løbende i udviklingen af programmet og kørt unit og integrationstest i slutningen af programmet. Dette har vi gjort med den hensigt at sørge for, at ikke kun brugeroplevelsen gennem brugergrænsefladen af programmet er så gnidningsfri som muligt, men også udviklingsprocessen ved at reducere den mængde bugs, og logiske fejl der kunne forekomme.
Grundet manglende erfaring med testning af programmer af dette omfang i gruppen, har der været forskellige forhindringer med testningen og udviklingen af dette, hvilket har gjort det til en læringsrig oplevelse. Vi har testet programmet på tre forskellige måder: manuel testning, unit tests, integrationstests. I gruppen har vi fælles prioriteret hvilke dele af testene som vi ville udvikle og implementere ud fra den ekspertise og den tilgængelige tid som vi har haft.

Vi har taget følgende beslutninger:

## Unit tests

Vi har implementeret unit tests, men i et begrænset omfang. Grundet programmets struktur og arkitektur har der været restriktioner i forhold til hvilke dele af programmet man har kunnet køre unit tests på.

Under udviklingen af UserServiceTest fandt vi ud af at det var muligt at skrive unit tests for validering funktionerne validateEmail, validatePassword, validatePhonenumber men at det ikke var muligt at lave unit tests for createUser og login. Dette skyldes at UserService selv laver sin egen UserMapper som er forbundet til den eksterne database. Da createUser og login begge gør brug af UserMapper som er forbundet til en database ville det ikke blive betragtet som en unit test hvis vi forsøgte at teste dem. Den samme logik er gældende for OrderService.

UserService.java

```java
package services;

import ...

public class UserService {  4 usages  Hannah Concillado +2
    private ConnectionPool connectionPool;  1 usage
    private UserMapper userMapper;  4 usages

    @Contract(pure = true)
    public UserService(ConnectionPool connectionPool) {  2 usages  Gabby AK +1
        this.connectionPool = connectionPool;
        this.userMapper = new UserMapper(connectionPool);
    }
```

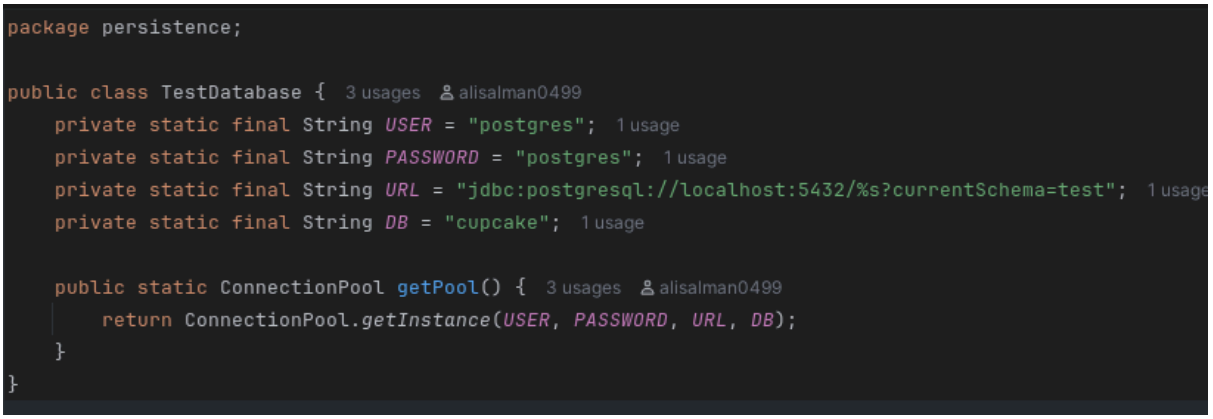
UserServiceTest.java

```java
class UserServiceTest {  Gabby AK

    private UserService userService;  4 usages

    @BeforeEach  Gabby AK
    void setUp() {
        ConnectionPool connectionPool = null;
        userService = new UserService(connectionPool);
    }

    @Test  Gabby AK
    void validateEmailWithoutSign() {
        assertThrows(IllegalUserDataException.class, ()-> userService.validateEmail("gabmail.com"));
    }

    @Test  Gabby AK
    void validatePasswordWithLessThanEightSigns() {
        assertThrows(IllegalUserDataException.class, ()-> userService.validatePassword("ab34"));

    }

    @Test  Gabby AK
    void validatePhonenumberWithAlphabet(){
        assertThrows(IllegalUserDataException.class, ()-> userService.validatePhoneNumber("abcdefgh"));
    }
```

Hvis vi skulle lave unit tests for de gældende services, ville løsningen være at omskrive UserService sådan at den kunne modtage en ”stub” mapper der havde faste data gennem ”constructor injection”. Vi så dette som en opgave uden for vores scope da det ville kræve at ændre arkitekturen og valgte i stedet at prioriterer vores tid og fokus på de andre tests, vi vil dog i fremtiden gøre overvejelser om fordelene og konsekvenserne ved at omskrive UserService på denne måde med fremtidige projekter.

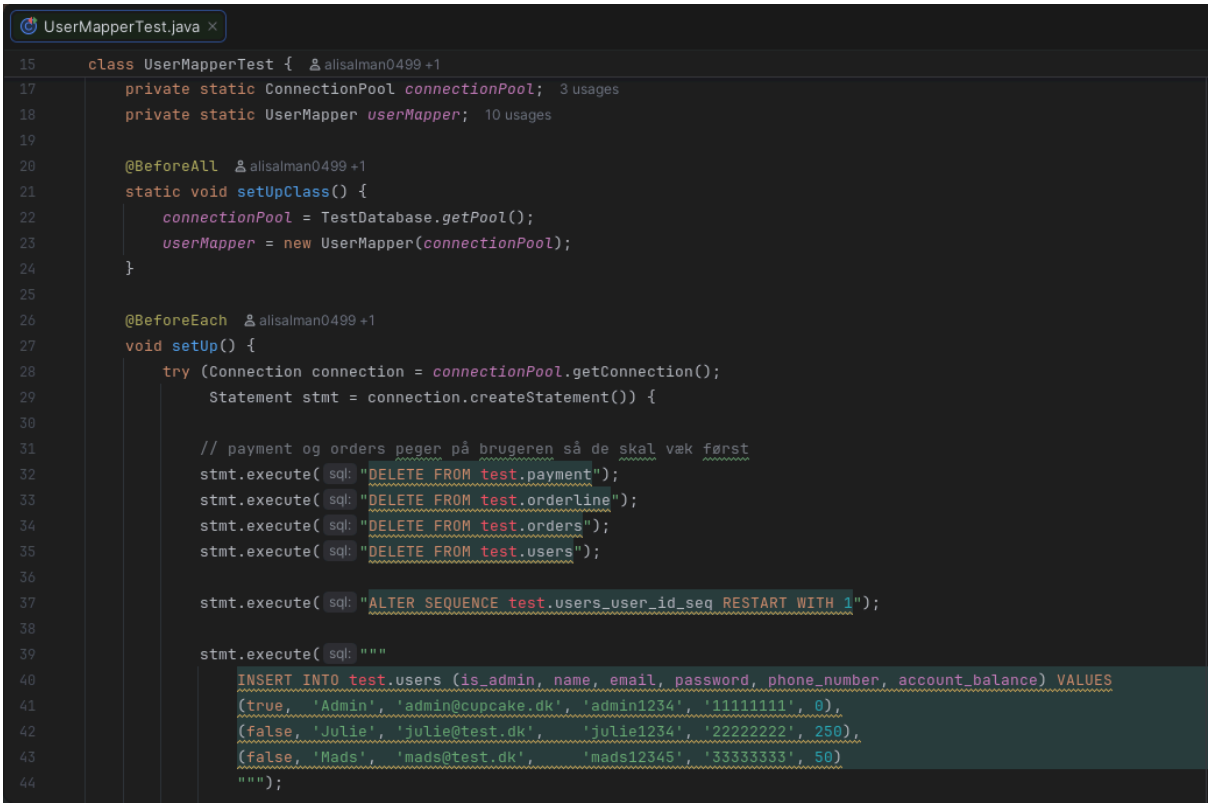
## Integrationstests

Vi har valgt at lave integrationsteste for alle vores Mappers. Til at starte med valgte vi at oprette en TestDatabase klasse som peger på et separat skema ”test” udelukkende til test kørsler sådan at vores ”main” database ikke får overskrevet sin data. TestDatabase opretter altså sin egen instans gennem ConnectionPool hvor at den peger på test skemaet i cupcake-databasen. Dette har også den fordel at vi ikke skal genskrive koden for at tale til test databasen i alle tre MapperTests men i stedet kan bruge TestDatabase til dette.



```
package persistence;

public class TestDatabase {  3 usages  alisalman0499
    private static final String USER = "postgres";  1 usage
    private static final String PASSWORD = "postgres";  1 usage
    private static final String URL = "jdbc:postgresql://localhost:5432/%s?currentSchema=test";  1 usage
    private static final String DB = "cupcake";  1 usage

    public static ConnectionPool getPool() {  3 usages  alisalman0499
        return ConnectionPool.getInstance(USER, PASSWORD, URL, DB);
    }
}
```

Før udførelsen af tests skal vi sørge for at test databasen er renset for tidligere test forsøg og derefter indsætte vores test data.



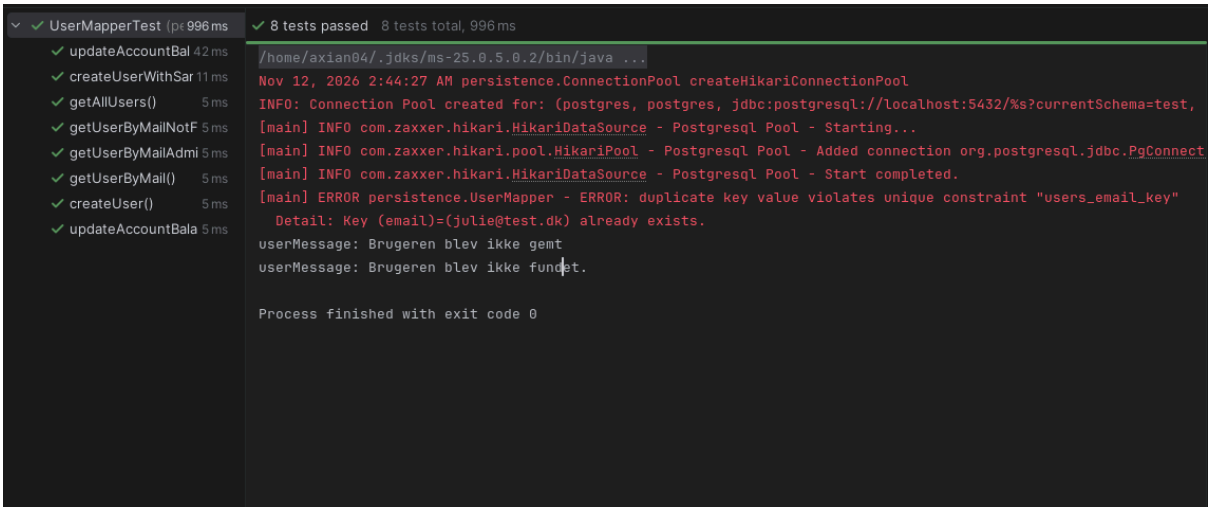

For selve testene valgte vi ikke kun at teste for når input er korrekt, men også når input er forkert eller handlingen ikke er gyldig. Som et eksempel har vi i UserMapperTest ikke kun lavet en test for createUser med en ny email men også en test for createUser med en allerede eksisterende email for at teste om programmet faktisk kaster fejl når brugeren foretager en handling der ikke burde være gyldig, på denne måde undgår vi kritiske fejl i programmet som eksempelvis to forskellige brugere med samme email og at programmet ikke går ned når der kommer en exception.

For vores getUserByMail metode har vi udover at teste for om brugeren eksisterer eller ikke eksisterer også sørget for at teste om programmet kan differentierer om brugeren er admin eller ej.

```java
@Test  alisalman0499
void getUserByMail() throws DatabaseException {
    User user = userMapper.getUserByMail("julie@test.dk");

    assertEquals( expected: 2, user.getUserId());
    assertEquals( expected: "Julie", user.getName());
    assertEquals( expected: "22222222", user.getPhoneNumber());
    assertEquals( expected: 250, user.getAccountBalance());
    assertFalse(user.isAdmin());
}

@Test  alisalman0499
void getUserByMailAdmin() throws DatabaseException {
    User user = userMapper.getUserByMail("admin@cupcake.dk");

    assertTrue(user.isAdmin());
}
```

Med vores test for updateAccountBalance, har vi dækket vores CREATE, READ, og UPDATE funktioner for databasen. Vi har ikke nået at implementere DELETE funktioner i det endelige program og eventuelle tests for dem.

Her er et eksempel på hvordan det ser ud, når alle testene for UserMapperTest kører korrekt i programmet.



### Manuel Testning

Den manuelle testning er der hvor vi primært har brugt det meste af vores tid når det gælder tests af programmet og det er også her vi har fundet størstedelen af fejlene og de mest kritiske. Som eksempel, tidligt i udviklingsprocessen opdagede vi at selvom vores kode i Java kommunikerede korrekt med databasen og at vi kunne oprette og hente bruger informationerne fra databasen, blev bruger informationen i realiteten ikke gemt i sessionen når brugerne faktisk loggede ind med gyldige informationer gennem hjemmesiden. Vi brugte også manuelle tests til at teste flowet af programmet i forhold til ordre, som for eksempel når en bruger prøvede at gå til betaling med ordre, hvad ser kunden faktisk? Her opdagede vi at siden fungerer korrekt hvis brugeren havde nok penge på saldoen men at hvis brugeren ikke havde tilstrækkeligt på kontoen ville siden der skulle informere dem om dette ikke bliver vist men i stedet sprunget over og brugeren ville blive sendt direkte til forsiden. Vores kode gemmer nu User objektet direkte i sessionen og ved for lav saldo gemmer vi en errorMessage i sessionen før en redirect til kurven. Begge fejl lå i controller og session lagene, hvilket vores automatiske tests ikke kunne fange. Disse scenarier tjener som et godt bevis på, hvorfor manuel testning også har sin plads i udviklingsprocessen.

## Status på implementation

Vi fik implementeret de fleste funktioner ind i systemet, både for kunder og for administratorer. Herunder gennemgår vi, hvad vi har nået at implementere, og hvad vi stadig mangler, med fokus på vores acceptkriterier og user stories (US).

### Kundens funktioner

Vi har implementeret muligheden for at oprette en bruger og logge ind (US-2 og US-4). Alle nyoprettede kunder starter ud med en saldo på 100 kr., som bruges til at kontrollere bekræftelse af ordre. Vi fik valideret email, telefonnummer og adgangskode. Der mangler dog stadig en del validering i adgangskoden, da vi kun har implementeret validering af længden og ikke validering af indholdet: bogstaver og tal (US-2).

Kunden kan vælge forskellige smagsvarianter af cupcakebunde og toppings samt angive antallet af cupcakes for de enkelte kombinationer (US-1 og US-3). Priserne bliver vist sammen med valgmulighederne, og den samlede pris bliver beregnet baseret på kundens valg. I indkøbskurven kan kunden ændre antallet af cupcakes ved hjælp af plus- og minusknapper. Ordrelinjen fjernes hvis den når til nul, og

indkøbskurven vil vise en besked om, at den er tom (US-3 og US-7). Ikke-registrerede brugere kan også tilføje cupcakes til indkøbskurven, men skal logge ind eller oprette en bruger, før ordren kan bekræftes.

Ved checkout kontrollerer systemet, om kunden har nok penge i deres saldo til at betale for ordren (US-1). Hvis der er nok penge, trækkes beløbet fra kundens konto, og ordren oprettes. Hvis der ikke er nok penge, får kunden en fejlbesked, og ordren oprettes ikke. Vi har valgt at betalingsprocessen går gennem en hardcodede saldo, og ikke gennem en betalingsfunktion.

## Administratorens funktioner

Vi har implementeret en administratorside, hvor administratoren kan se en oversigt over ordrer, som inkluderer ordrenummer, dato, ordrestatus, betalingsstatus, kundens email, samlet pris og detaljer (US-5). Administratoren kan også åbne en ordre ved at klikke på ‘se ordre' for at se dens ordrelinjer og ændre ordrestatus (US-5 og US-6). Derudover kan administratoren også se en oversigt over alle kunder, der har afgivet en bekræftet ordre, med deres navne og emailadresser (US-6).

Det er ikke muligt for administratoren at åbne en bestemt kunde og se alle kundens tidligere ordrer (US-6). Det er heller ikke muligt at slette ordrer (US-8).

## Sider og design

De fleste af de centrale sider fra navigationsdiagrammet er blevet implementeret, herunder forsiden, login, brugeroprettelse, indkøbskurven, profilsiden og administratorsiden. Vi har også lavet en fælles CSS-fil og genbrugt header og footer på flere af siderne, så de har nogenlunde samme udseende.

Vi har desuden lavet en ekstra færdiglavet cupcake-oversigtsside, som ikke var en del af opgaven, og derfor ikke videreudviklet. Siden kan godt benyttes, men funktionen til at tilføje cupcakes til indkøbskurven virker ikke.

## Fravalgte og manglende funktioner

Vi har valgt ikke at implementere US-2b, som omhandler muligheden for at gemme ordrer til senere betaling. Denne user story blev derfor fravalgt undervejs i projektet.

I US-2 mangler der stadig en kontrol af, at adgangskoden skal indeholde bogstaver og tal, da man kan indtaste en adgangskode med kun specialtegn. Vi fik heller ikke opfyldt alle acceptkriterierne i US-6, da administratoren ikke kunne se den enkelte kundes tidligere ordrer. Vi nåede heller ikke at implementere US-8, som omhandler

administratorens mulighed for at slette ordrer. Der mangler både en slettefunktion og en bekræftelse inden sletning.

Alt i alt nåede vi at lave syv ud af otte user stories, hvor nogle af de user stories er delvist gennemført.

### Fremtidig implementation

Vi ville arbejde videre med at strukturere koden bedre og omformulere logikken på ordrebekræftelse siden. Derudover ville vi gennemgå koden for at fjerne død og overflødig kode samt gentagne funktioner.

Vi har desuden en "payment" -tabel i ER-diagrammet, men grundet prioriteringer og tidspres har vi ikke haft mulighed for at implementere den.

Vi havde overvejet at styrke login-sikkerheden yderligere, del med ekstra validering af adgangskoden, så den skulle indeholde et bestemt antal bogstaver og tal, samt ved at hashe brugeren adgangskoder inden de gemmes i databasen. Begge dele ville have været gode sikkerhedsfunktioner, men på grund af tidspres valgte vi at prioritere at gennemføre vores user stories og lod valideringen være minimal.

## Proces

### Planer for arbejdsform og projektforløb

Vi startede med en tidsplan, en teamkontrakt og at udarbejde acceptkriterier til vores user stories. Vi aftalte hvordan vi ville prioritere user stories, at arbejde i feature branches ud fra et kanban-board og at blive enige om navngivning og om hvornår vi skulle merge og teste.

### Forløbet i praksis

De første dage gik med analyse og design af domænemodellen, ERD og mockups, inden vi begyndte at kode. Teamet arbejdede primært på samme tid online, så vi kunne kommunikere live, men når der var behov for at arbejde på forskellige tidspunkter, holdt vi hinanden opdateret over Discord.
Vores Tech lead samlede branches på main som den eneste, hvor resten af teamet arbejdede på branches ud fra og dagligt pushet til.

# Refleksion

## Arbejdsform og værktøjer

Vi brugte GitHub Projects med et fælles kanban-board, hvor user stories blev delt op i tasks.  Det gav os overblik over hvad vi ville nå og hvor langt vi var.  Hver eneste dag startede vi med et morgenmøde, hvor vi samlede op på hvor langt vi var, opdaterede kanban-boardet, lagde en plan for dagen og snakkede om eventuelle forhindringer, både i koden og i det virkelige liv. Hver opgave blev lavet i sin egen feature branch, og vi committede og pushede dagligt. Efter morgenmødet kunne alle starte ud fra en opdateret main.

## Roller

Ali var tech lead og stod for at merge alle branches til main og holde overblik over systemet. Gabriella og Hannah var product owners og holdt vores beslutninger op mod opgavebeskrivelsen og user stories. Kristina var scrum master og stod for morgenmøderne, holdt kanban-boardet opdateret og sikrede at alle havde en opgave, så projektet hele tiden var i udvikling.

## Kodestandard og DoD

Vi aftalte tidligt hvordan variabler og metoder skulle navngives og hvornår vi skulle merge og teste. Vi blev enige om følgende DoD:

- Alle acceptkriterier i story'en er opfyldt.
- Det virker fra start til slut i browseren, fx fra forsiden til kurven til betaling.
- Data gemmes og hentes korrekt fra databasen hvor det er relevant.
- Koden er committet til git
- Vi har udført dokumenteret test med fx JUnit (*Nået vi ikke at overholde løbende*)

## Hvad gik godt og hvad har vi lært?

Vi brugte god tid på analysen og det gav os en fælles forståelse af systemet og databasen før vi begyndte at kode. Vi var derfor generelt enige om hvordan koden skulle laves. Ulempen var, at vi kom senere i gang med at kode og ikke nåede i mål med alle user stories. Samtidig dækkede analysen domænet og databasen godt, men ikke strukturen i koden. Næste gang vil vi også lave et klassediagram inden vi begynder at kode. Det ville have hjulpet os med at undgå død kode, der ligger ubrugt i projektet fordi vi på forhånd kunne have planlagt, hvilke metoder der skulle bruges samt i hvilke klasser de skulle ligge.

Vores samarbejde er gået rigtig godt. Vi har haft en god kommunikation, alle har overholdt vores aftaler, og alle har haft en følelse over projektet og af at blive hørt.

En anden ting vi kunne have gjort bedre var at indlægge fysiske møder hvor vi gennemgår kode sammen eller sammen kigger på komplekse eller centrale dele af koden, så vi har mulighed for at snakke al koden ordentligt igennem og opnå endnu bedre forståelse for flow og systemet i detaljer.